

TRƯỜNG ĐẠI HỌC KHOA HỌC

KHOA CÔNG NGHỆ THÔNG TIN

CHUYÊN NGÀNH MẠNG MÁY TÍNH



TIỂU LUẬN

Học phần: Phát triển ứng dụng IoT - Nhóm 4

ĐỀ TÀI: Điều khiển rèm cửa tự động với ESP32

Giáo viên hướng dẫn: Võ Việt Dũng

MỤC LỤC	TRANG
LỜI MỞ ĐẦU	3
I. Mục tiêu đề tài	4
1. Mục tiêu tổng quát.....	4
2. Mục tiêu cụ thể	4
II. Nội dung chính	5
1. Thông tin thiết bị.....	5
1.1 ESP32	5
1.2. wokwi-a4988 Reference	6
1.3. wokwi-stepper-motor Reference	7
1.4 Nền tảng Telegram BOT.....	8
2. Sơ đồ hệ thống.....	8
3. Quy trình thực hiện.....	8
4. Công cụ sử dụng.....	10
5. Kết quả thực hiện.....	11
III. Kết luận	12
1. Công việc thực hiện.....	12
2. Phương hướng phát triển.....	12
IV. Mã nguồn	13
1. Mã nguồn mô phỏng.....	13
2. Mã nguồn logic.....	13
V. Tài liệu tham khảo	19

Lời mở đầu

Trong bối cảnh khoa học công nghệ ngày càng phát triển mạnh mẽ, xu hướng ứng dụng các hệ thống tự động hóa vào đời sống sinh hoạt đang trở nên phổ biến và giữ vai trò quan trọng trong việc nâng cao chất lượng cuộc sống. Đặc biệt, cùng với sự phát triển của Internet vạn vật (IoT), các thiết bị gia dụng thông minh không chỉ giúp con người tiết kiệm thời gian, công sức mà còn mang lại sự tiện nghi, an toàn và hiệu quả trong quá trình sử dụng. Trong số đó, hệ thống rèm cửa tự động là một trong những ứng dụng tiêu biểu, góp phần hiện đại hóa không gian sống và làm việc.

Xuất phát từ nhu cầu thực tiễn đó, đề tài **“Điều khiển rèm cửa tự động với ESP32 sử dụng động cơ bước, điều khiển qua ứng dụng telegram”** được lựa chọn nhằm nghiên cứu và xây dựng một mô hình điều khiển rèm cửa thông minh có khả năng đóng, mở linh hoạt từ xa thông qua thiết bị di động. Việc sử dụng vi điều khiển ESP32 giúp hệ thống dễ dàng kết nối mạng không dây, hỗ trợ giao tiếp với ứng dụng điều khiển một cách thuận tiện. Bên cạnh đó, động cơ bước được lựa chọn làm cơ cấu chấp hành nhờ khả năng điều khiển chính xác về góc quay và vị trí, phù hợp với yêu cầu đóng mở rèm theo từng mức độ khác nhau.

Thông qua đề tài này, người thực hiện không chỉ tìm hiểu nguyên lý hoạt động của hệ thống nhúng, truyền thông không dây và điều khiển động cơ, mà còn có cơ hội tiếp cận với quy trình thiết kế, mô phỏng và triển khai một ứng dụng thực tế trong lĩnh vực nhà thông minh. Đồng thời, đề tài cũng góp phần củng cố kiến thức chuyên môn, rèn luyện kỹ năng lập trình, tư duy thiết kế hệ thống và khả năng ứng dụng lý thuyết vào thực tiễn.

Từ đó, đề tài hướng đến mục tiêu xây dựng một mô hình rèm cửa tự động có tính ứng dụng cao, hoạt động ổn định, dễ triển khai và có thể phát triển mở rộng trong các hệ thống điều khiển thông minh trong tương lai.

I. Mục tiêu đề tài

1. Mục tiêu tổng quát:

Nghiên cứu, thiết kế và thi công mô hình hệ thống điều khiển rèm cửa tự động dựa trên nền tảng Internet of Things (IoT). Hệ thống sử dụng vi điều khiển ESP32 làm bộ xử lý trung tâm, động cơ bước để vận hành cơ khí và cho phép người dùng điều khiển, giám sát từ xa thông qua ứng dụng Telegram.

2. Mục tiêu cụ thể

- **Về phần cứng:** Tìm hiểu, lựa chọn và kết nối thành công các linh kiện bao gồm vi điều khiển ESP32, mạch điều khiển động cơ (driver) và động cơ bước. Đảm bảo hệ thống mạch điện hoạt động ổn định, cung cấp đủ dòng và áp cho động cơ kéo được rèm.

- **Về phần mềm:** Lập trình cho ESP32 kết nối với mạng Wi-Fi và giao tiếp với hệ thống Telegram Bot API. Xây dựng thuật toán điều khiển động cơ bước quay chính xác (tính toán số bước/góc quay) để mô phỏng đúng hành trình đóng/mở của rèm cửa.

- **Về giao diện và ứng dụng:** Xây dựng và cấu hình một Telegram Bot trực quan. Thiết lập các lệnh điều khiển cơ bản (ví dụ: /mo, /dong, /dung) để người dùng có thể dễ dàng tương tác với rèm cửa ở bất kỳ đâu có kết nối Internet.

- **Về kiểm thử và đánh giá:** Lắp ráp mô hình thực tế, đo lường độ trễ (latency) của tín hiệu điều khiển từ ứng dụng đến thiết bị. Đánh giá độ ổn định của hệ thống khi hoạt động liên tục và xử lý các lỗi phát sinh (nếu có).

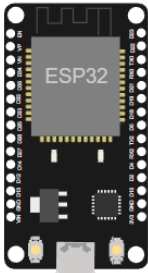
II. Nội dung chính

1. Thông tin thiết bị

1.1 ESP32 (Vi điều khiển trung tâm):

ESP32 Simulation

The ESP32 is a popular WiFi and Bluetooth-enabled microcontroller, widely used for IoT Projects. Wokwi simulates the ESP32, ESP32-C3, ESP32-S2, ESP32-S3, ESP32-C5 (alpha), ESP32-C6, ESP32-H2, and ESP32-P4 (beta).



Trong Wokwi, ESP32 được mô phỏng dựa trên dòng vi điều khiển lõi kép mạnh mẽ của Espressif. Đây là "bộ não" của hệ thống, chịu trách nhiệm xử lý logic và kết nối mạng.

- **Kết nối Wi-Fi:** Wokwi hỗ trợ mô phỏng kết nối Wi-Fi qua một mạng ảo có tên là Wokwi-GUEST. Tính năng này cho phép bạn viết code kết nối với Telegram Bot API và mô phỏng việc gửi/nhận dữ liệu Internet y như thiết bị thật.

- **Chân GPIO (General Purpose Input/Output):** Cung cấp các chân để giao tiếp với các module ngoại vi.

+ Để điều khiển A4988, bạn có thể chọn các chân Digital Output bất kỳ trên ESP32 để cấp tín hiệu STEP và DIR.

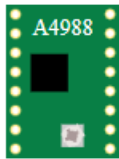
+ Nguồn điện logic 3.3V từ ESP32 sẽ được dùng để cấp cho mạch driver A4988 (chân VDD).

- **Hiệu suất:** Tốc độ xung nhịp 240MHz, dư sức để vừa xử lý các tác vụ mạng (nhận lệnh từ Telegram) vừa băm xung (PWM/cấp xung) điều khiển động cơ bước mà

1.2. wokwi-a4988 Reference (Mạch điều khiển động cơ)

wokwi-a4988 Reference

A4988 Stepper Motor driver, for use with [wokwi-stepper-motor](#)



A4988 là module driver (trình điều khiển) rất phổ biến dùng để trung gian giữa vi điều khiển (ESP32) và động cơ bước. Do ESP32 không thể cấp đủ dòng điện áp lớn để chạy trực tiếp động cơ, A4988 sẽ đóng vai trò khuếch đại tín hiệu và điều tiết dòng điện.

Các chân kết nối quan trọng (Pinout):

- Giao tiếp với ESP32:

- + STEP: Nhận xung (pulse) từ ESP32. Mỗi xung tương ứng với một bước quay của động cơ.

- + DIR: Nhận tín hiệu (HIGH/LOW) từ ESP32 để xác định chiều quay (ví dụ: kéo rèm lên hoặc hạ rèm xuống).

- Kết nối Nguồn:

- + VDD & GND (Logic): Kết nối với 3.3V và GND của ESP32.

- + VMOT & GND (Motor): Kết nối với nguồn cấp ngoài mạnh hơn (thường từ 8V - 35V) để cung cấp lực kéo cho động cơ.

- Giao tiếp với Động cơ bước:

- + 1A, 1B: Nối vào cuộn dây thứ nhất của động cơ.

- + 2A, 2B: Nối vào cuộn dây thứ hai của động cơ.

- Điều khiển vi bước (Microstepping):

- + Gồm các chân MS1, MS2, MS3. Tùy theo cách cấu hình (kết nối với HIGH hoặc LOW), bạn có thể chia nhỏ một bước của động cơ

thành 1/2, 1/4, 1/8, hoặc 1/16 bước. Việc này giúp động cơ chạy êm hơn và định vị chính xác hơn.

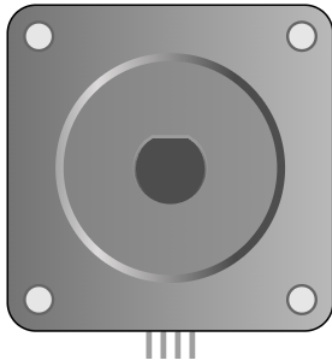
- **Các chân trạng thái:** ENABLE (kích hoạt module), SLEEP và RESET (thường nối SLEEP với RESET vào nhau để mạch hoạt động bình thường).

1.3. wokwi-stepper-motor Reference (Động cơ bước)

🏠 > Diagram Reference > wokwi-stepper-motor

wokwi-stepper-motor Reference

A bipolar stepper motor



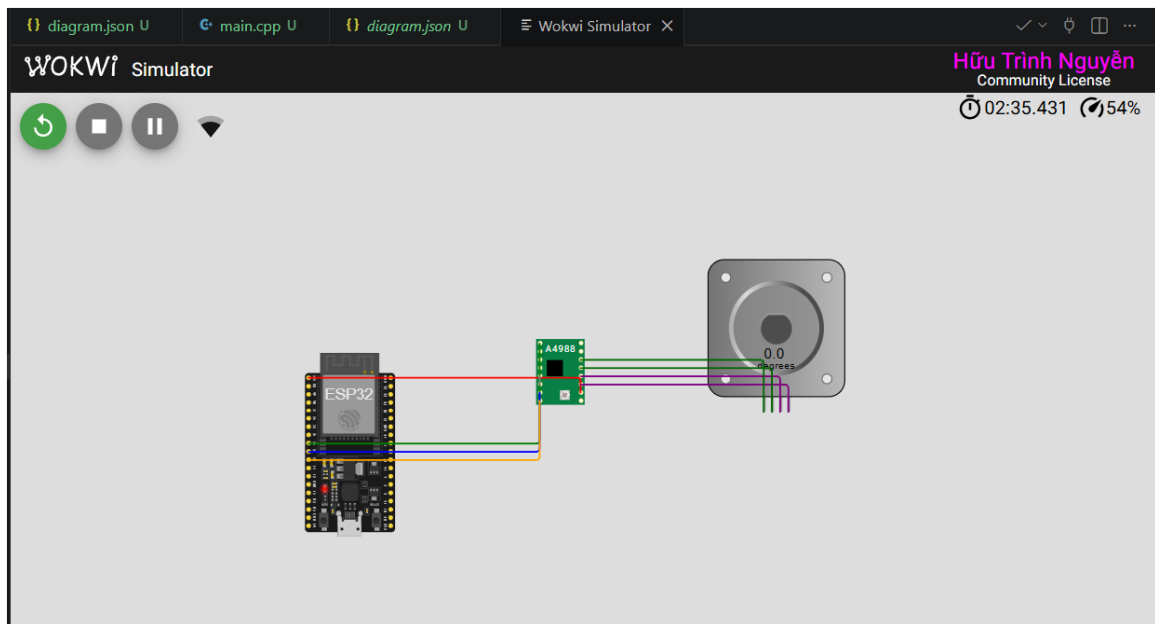
Wokwi mô phỏng một động cơ bước lưỡng cực (bipolar stepper motor) tiêu chuẩn, loại thường thấy nhất là NEMA 17 được dùng nhiều trong rèm cửa tự động và máy in 3D.

- **Cấu tạo và Kết nối:** Động cơ có 4 dây chia thành 2 cặp tương ứng với 2 cuộn dây (Coil A và Coil B). Bốn dây này (A+, A-, B+, B-) sẽ được nối trực tiếp vào các chân đầu ra của module A4988.
- **Góc bước (Step Angle):** Mặc định, động cơ bước trong mô phỏng thường có 200 bước cho mỗi vòng quay tròn (360 độ). Điều này có nghĩa là mỗi bước (step) sẽ làm trục động cơ quay một góc 1.8 độ.
- **Cơ chế hoạt động:** Bằng cách đếm số lượng xung truyền từ ESP32 qua A4988 tới động cơ, bạn có thể kiểm soát chính xác 100% vị trí của rèm cửa. Ví dụ: Nếu biết rèm cần quay 10 vòng để đóng hoàn toàn, bạn chỉ cần xuất ra chính xác $10 \times 200 = 2000$ xung.

1.4 Nền tảng Telegram BOT

Hoạt động như một công cụ truyền thông tin từ xa. Thông qua Telegram Bot API, hệ thống có thể gửi yêu cầu hoạt động cho rèm cửa một cách nhanh chóng, miễn phí và bảo mật.

2. Sơ đồ hệ thống



- Sử dụng wokwi để tạo môi trường mô phỏng cho các thiết bị trước khi thực hiện thực tế

3. Quy trình thực hiện

Quy trình triển khai hệ thống IoT được thực hiện theo các bước sau:

Bước 1: Chuẩn bị môi trường phát triển gồm VS Code, PlatformIO, tiện ích mở rộng Wokwi và thiết lập Bot trên ứng dụng Telegram.

Bước 2: Thiết kế và mô phỏng mạch trên Wokwi với các linh kiện: ESP32, wokwi-stepper-motor Reference (Động cơ bước), wokwi-a4988 Reference (Mạch điều khiển động cơ)

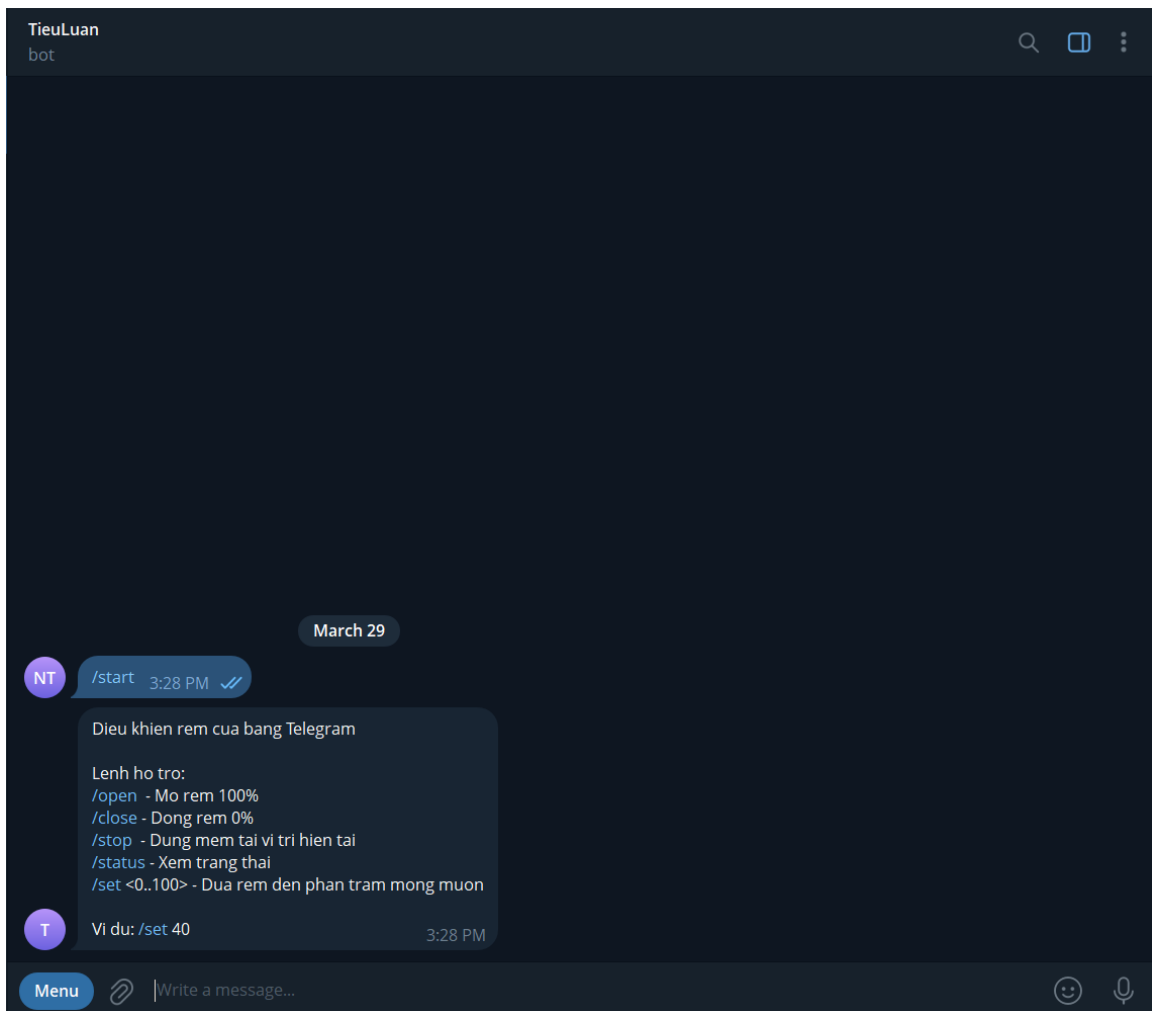
Bước 3: Lập trình ESP32 với ngôn ngữ C++ để:

Kết nối mạng không dây: Wifi – Telegram config

Khởi tạo kết nối HTTPS bảo mật với Telegram Bot API.

Điều khiển trực quay của wokwi-stepper-motor Reference (Động cơ bước)
để thực hiện đóng mở rèm cửa

Bước 4: Cấu hình Telegram BOT



Bước 5: Tích hợp trình mô phỏng Wokwi vào VS Code thông qua file cấu hình diagram.json và wokwi.toml.

Bước 6: Kiểm thử hệ thống: Thực hiện các bài test để đánh giá tốc độ nhận tin nhắn trên Telegram và hoạt động đóng mở tron tru

4. Công cụ sử dụng

Arduino Framework: Khung lập trình tiêu chuẩn dùng để viết mã nguồn cho vi điều khiển ESP32.

Visual Studio Code: Trình soạn thảo mã nguồn chuyên nghiệp, nơi quản lý toàn bộ dự án.

PlatformIO: Tiện ích hỗ trợ quản lý thư viện (UniversalTelegramBot, ArduinoJson), biên dịch mã nguồn và nạp chương trình.

Wokwi Simulator: Nền tảng mô phỏng mạch điện tử và kết nối WiFi ảo, giúp kiểm tra hệ thống mà không cần phần cứng thật.

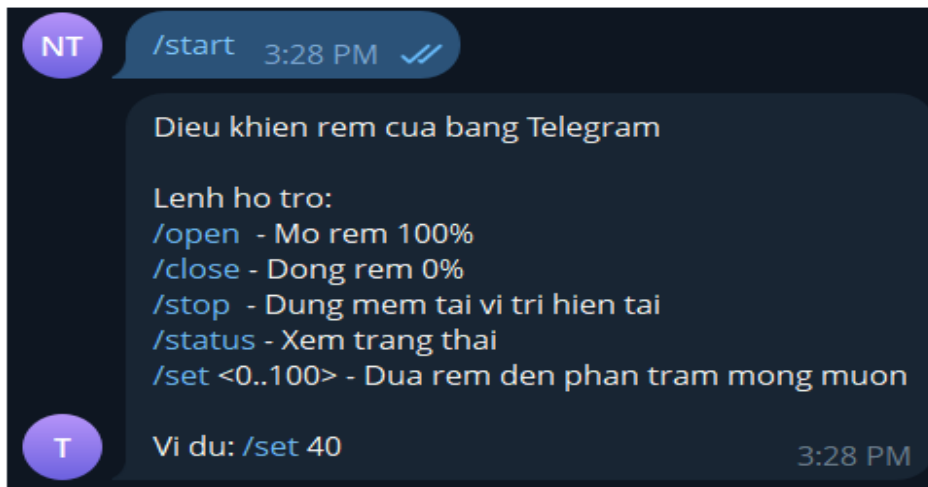
Telegram Bot API: Cổng giao tiếp trung gian, cho phép người dùng điều khiển ESP32 trực tiếp đến ứng dụng Telegram của người dùng.

Thư viện UniversalTelegramBot: Thư viện mã nguồn mở giúp đơn giản hóa việc gửi/nhận tin nhắn qua Telegram API.

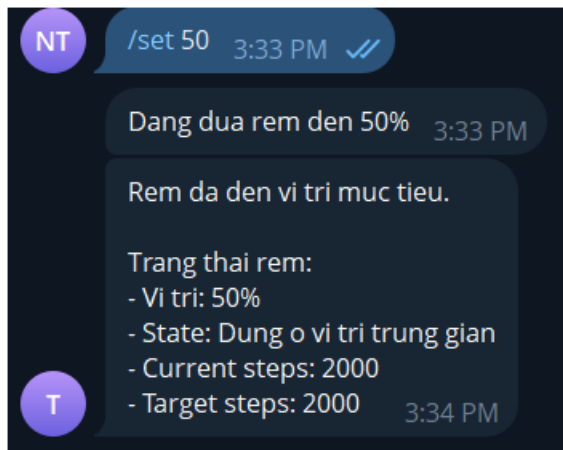
Thư viện ArduinoJson: Dùng để đóng gói và xử lý các chuỗi dữ liệu JSON khi giao tiếp với máy chủ Telegram.

5. Kết quả thực hiện

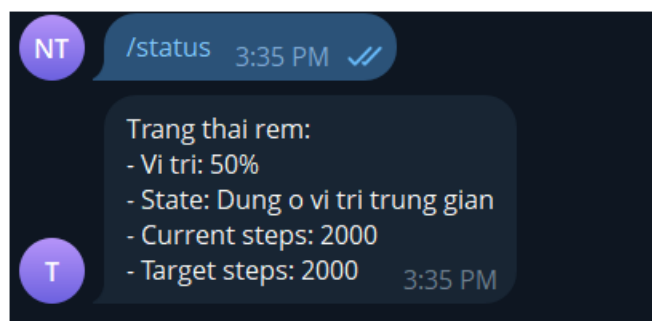
- Khi khởi động hệ thống nhập /start để khởi động



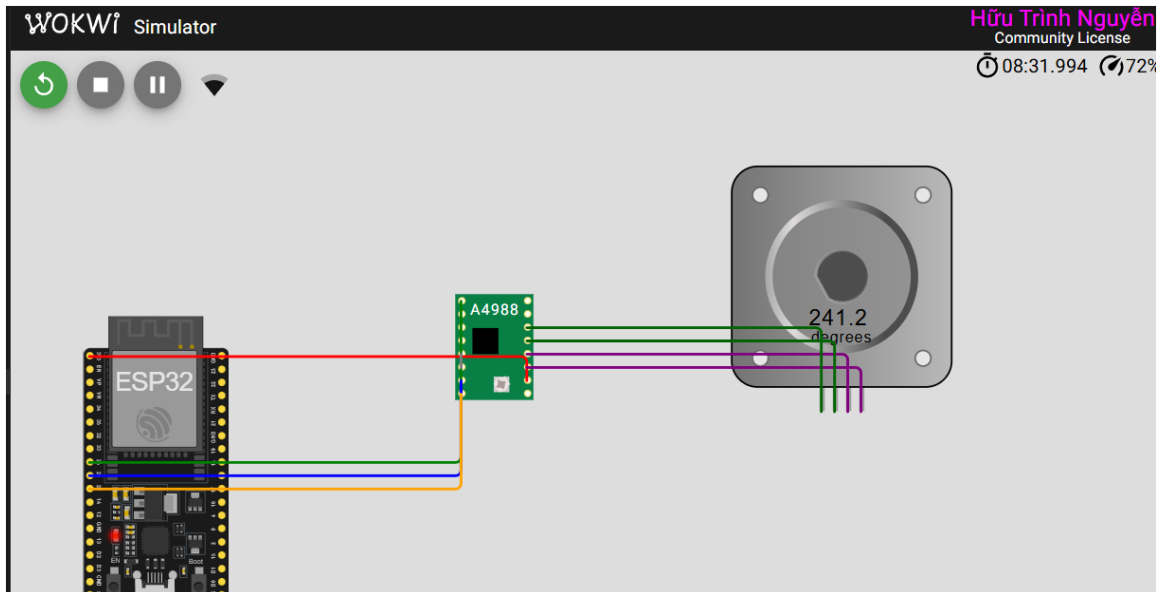
- /Set 50 để mở 1 nửa rèm



Nhập /status để xem vị trí rèm hiện tại



Mô phỏng Wokwi



+ Khi nhập lệnh, tele sẽ gửi yêu cầu cho ESP32 từ đó chuyển sang mạch trung gian và truyền tới motor, motor xoay thì rèm cửa hoạt động đóng-mở

III. Kết luận

1. Công việc thực hiện

- Trong quá trình thực hiện đề tài đã hoàn thành các nội dung chính sau:

Xây dựng thành công hệ thống IoT thực hiện việc đóng mở rèm cửa bằng lệnh từ Telegram

Thiết kế mô phỏng mạch điện tử trên Wokwi

Lập trình thành công vi điều khiển motor quay theo yêu cầu

Thiết lập giao thức HTTPS để giao tiếp với Telegram Bot API

Kiểm thử hệ thống hoạt động ổn định, tin nhắn gửi về điện thoại có độ trễ thấp

2. Phương hướng phát triển:

Trong tương lai đề tài có thể phát triển theo phương hướng như sau:

Đóng mở rèm tự động với cảm biến ánh sáng wokwi-photoresistor-sensor mỗi khi cảm biến nhận được nhiệt độ, ánh sáng quá lớn thì sẽ tự động đóng và thông báo về Telegram và khi nhận nhiệt độ, ánh sáng ít thì tự động mở rèm

Phát triển giao diện giám sát trực quan hơn trên Web hoặc Mobile App để theo dõi
Tích hợp thêm module SIM để gọi điện hoặc nhắn tin SMS trực tiếp trong trường
hợp mất kết nối WiFi, Nâng cao bảo mật và xác thực người dùng để tránh việc
điều khiển nhầm lẫn từ bên ngoài

IV. Mã nguồn

1. Mã nguồn mô phỏng

```
{} diagram.json • PIO Home main.cpp {} diagram.json • wokwi.toml ✓
TieuLuan > {} diagram.json > {} parts > {} 2
1 {
2   "version": 1, "author": "Nguyen Huu Trinh", "editor": "wokwi",
3   "parts": [
4     { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": {} },
5     { "type": "wokwi-a4988", "id": "drv", "top": -16, "left": 270, "attrs": {} },
6     { "type": "wokwi-stepper-motor", "id": "motor", "top": -110, "left": 470, "attrs": { "size": "17",
7       "display": "angle"
8     }
9   ]
10 },
11 ],
12 "connections": [
13   [ "esp:TX", "$serialMonitor:RX", "", [ ] ],
14   [ "esp:RX", "$serialMonitor:TX", "", [ ] ],
15   [ "esp:3V3", "drv:VDD", "red", [ ] ],
16   [ "esp:GND.1", "drv:GND", "black", [ ] ],
17   [ "esp:25", "drv:ENABLE", "green", [ ] ],
18   [ "esp:26", "drv:STEP", "blue", [ ] ],
19   [ "esp:27", "drv:DIR", "orange", [ ] ],
20   [ "drv:RESET", "drv:SLEEP", "gray", [ ] ],
21   [ "drv:1A", "motor:B+", "purple", [ ] ],
22   [ "drv:1B", "motor:B-", "purple", [ ] ],
23   [ "drv:2A", "motor:A+", "darkgreen", [ ] ],
24   [ "drv:2B", "motor:A-", "darkgreen", [ ] ]
25 ],
26 "dependencies": {}
27 }
```

2. Mã nguồn logic

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>
#include <ArduinoJson.h>
```

```

#include <AccelStepper.h>
#include <time.h>

const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";
const char* BOT_TOKEN = "8034409980:AAEGnnHqDMRA4Zwqmp1bArb-FDIgjBQmrDE";
const char* ALLOWED_CHAT_ID = "7999542154";
static const uint8_t PIN_STEP = 26;
static const uint8_t PIN_DIR = 27;
static const uint8_t PIN_EN = 25;
static const long CURTAIN_FULL_OPEN_STEPS = 4000;
static const float MAX_SPEED = 1200.0;
static const float ACCELERATION = 700.0;
WiFiClientSecure securedClient;
UniversalTelegramBot bot(BOT_TOKEN, securedClient);
AccelStepper stepper(AccelStepper::DRIVER, PIN_STEP, PIN_DIR);
unsigned long lastBotCheck = 0;
const unsigned long BOT_POLL_INTERVAL = 700;
const long CLOSED_POS = 0;
const long OPEN_POS = CURTAIN_FULL_OPEN_STEPS;
String activeChatId = "";
bool notifyWhenArrive = false;
bool wasMoving = false;
long clampLong(long value, long low, long high) {
    if (value < low) return low;
    if (value > high) return high;
    return value;
}

int getPositionPercent() {
    long pos = clampLong(stepper.currentPosition(), CLOSED_POS, OPEN_POS);
    return map(pos, CLOSED_POS, OPEN_POS, 0, 100);
}

String getCurtainState() {
    if (stepper.distanceToGo() != 0) return "Dang di chuyen";
    int percent = getPositionPercent();
    if (percent <= 2) return "Dang dong";
    if (percent >= 98) return "Dang mo";
    return "Dung o vi tri trung gian";
}

String buildStatusMessage() {
    String msg = "Trang thai rem:\n";
    msg += "- Vi tri: " + String(getPositionPercent()) + "%\n";

```

```

    msg += "- State: " + getCurtainState() + "\n";
    msg += "- Current steps: " + String(stepper.currentPosition()) + "\n";
    msg += "- Target steps: " + String(stepper.targetPosition());
    return msg;
}

void moveToPercent(int percent, const String& chatId) {
    percent = constrain(percent, 0, 100);
    long target = map(percent, 0, 100, CLOSED_POS, OPEN_POS);
    stepper.moveTo(target);
    activeChatId = chatId;
    notifyWhenArrive = true;
}

void sendHelp(const String& chatId) {
    String msg;
    msg += "Dieu khien rem cua bang Telegram\n\n";
    msg += "Lenh ho tro:\n";
    msg += "/open - Mo rem 100%\n";
    msg += "/close - Dong rem 0%\n";
    msg += "/stop - Dung mem tai vi tri hien tai\n";
    msg += "/status - Xem trang thai\n";
    msg += "/set <0..100> - Dua rem den phan tram mong muon\n\n";
    msg += "Vi du: /set 40";
    bot.sendMessage(chatId, msg, "");
}

void connectWiFi() {
    Serial.println("Dang ket noi WiFi...");
    WiFi.mode(WIFI_STA);

    // Tối ưu cho Wokwi
    if (String(WIFI_SSID) == "Wokwi-GUEST") {
        WiFi.begin(WIFI_SSID, WIFI_PASSWORD, 6);
    } else {
        WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    }

    while (WiFi.status() != WL_CONNECTED) {
        delay(300);
        Serial.print(".");
    }

    Serial.println("\nWiFi connected!");
    Serial.print("IP: ");

```

```

    Serial.println(WiFi.localIP());
}

void syncTimeForTLS() {
    Serial.print("Dong bo thoi gian NTP");
    configTime(0, 0, "pool.ntp.org");

    time_t now = time(nullptr);
    while (now < 24 * 3600) {
        delay(200);
        Serial.print(".");
        now = time(nullptr);
    }

    Serial.println("\nTime synced.");
}

bool isAuthorizedChat(const String& chatId) {
    if (strlen(ALLOWED_CHAT_ID) == 0) {
        return true; // Cho phép mọi chat nếu chưa khoá
    }
    return chatId == String(ALLOWED_CHAT_ID);
}

void handleCommand(const String& chatId, String text, const String&
fromName) {
    text.trim();

    Serial.println("==== Telegram message ====");
    Serial.println("From: " + fromName);
    Serial.println("chat_id: " + chatId);
    Serial.println("text: " + text);

    if (!isAuthorizedChat(chatId)) {
        bot.sendMessage(chatId, "Ban khong duoc phep dieu khien thiet bi
nay.", "");
        return;
    }

    if (text == "/start" || text == "/help") {
        sendHelp(chatId);
        return;
    }

    if (text == "/open") {

```

```

        moveToPercent(100, chatId);
        bot.sendMessage(chatId, "Dang mo rem...", "");
        return;
    }

    if (text == "/close") {
        moveToPercent(0, chatId);
        bot.sendMessage(chatId, "Dang dong rem...", "");
        return;
    }

    if (text == "/stop") {
        stepper.stop(); // dung mem, co giam toc
        activeChatId = chatId;
        notifyWhenArrive = true;
        bot.sendMessage(chatId, "Dang dung rem...", "");
        return;
    }

    if (text == "/status") {
        bot.sendMessage(chatId, buildStatusMessage(), "");
        return;
    }

    if (text.startsWith("/set ")) {
        String value = text.substring(5);
        value.trim();
        int percent = value.toInt();

        if (percent < 0 || percent > 100) {
            bot.sendMessage(chatId, "Gia tri khong hop le. Hay dung /set
<0..100>", "");
            return;
        }

        moveToPercent(percent, chatId);
        bot.sendMessage(chatId, "Dang dua rem den " + String(percent) + "%",
        "");
        return;
    }

    bot.sendMessage(chatId, "Lenh khong hop le. Gui /start de xem huong
dan.", "");
}

```

```

void handleNewMessages(int numNewMessages) {
    for (int i = 0; i < numNewMessages; i++) {
        String chatId = bot.messages[i].chat_id;
        String text = bot.messages[i].text;
        String fromName = bot.messages[i].from_name;

        handleCommand(chatId, text, fromName);
    }
}

void setup() {
    Serial.begin(115200);
    Serial.println("\nKhoi dong he thong rem cua...");

    pinMode(PIN_EN, OUTPUT);
    digitalWrite(PIN_EN, LOW); // A4988 ENABLE active-low => LOW = enable

    stepper.setMaxSpeed(MAX_SPEED);
    stepper.setAcceleration(ACCELERATION);

    // Quy ước khi khởi động: rèm đang đóng hoàn toàn
    // Ngoài thực tế nên thêm công tắc hành trình để homing
    stepper.setCurrentPosition(CLOSED_POS);

    connectWiFi();

    securedClient.setCACert(TELEGRAM_CERTIFICATE_ROOT);
    syncTimeForTLS();

    bot.setMyCommands(
        "[
        {\"command\": \"start\", \"description\": \"Xem huong dan\"},
        {\"command\": \"open\", \"description\": \"Mo rem\"},
        {\"command\": \"close\", \"description\": \"Dong rem\"},
        {\"command\": \"stop\", \"description\": \"Dung rem\"},
        {\"command\": \"status\", \"description\": \"Trang thai rem\"}
        ]"
    );

    Serial.println("He thong san sang. Hay nhan /start tren Telegram.");
}

void loop() {
    // Quan trọng: phải gọi liên tục để stepper chạy mượt
    stepper.run();
}

```

```

if (millis() - lastBotCheck > BOT_POLL_INTERVAL) {
    int numNewMessages = bot.getUpdates(bot.last_message_received + 1);

    while (numNewMessages) {
        handleNewMessages(numNewMessages);
        numNewMessages = bot.getUpdates(bot.last_message_received + 1);
    }

    lastBotCheck = millis();
}

bool isMoving = (stepper.distanceToGo() != 0);

if (wasMoving && !isMoving && notifyWhenArrive && activeChatId.length()
> 0) {
    bot.sendMessage(activeChatId, "Rem da den vi tri muc tieu.\n\n" +
buildStatusMessage(), "");
    notifyWhenArrive = false;
}

wasMoving = isMoving;
}

```

V. Tài liệu tham khảo:

Tài liệu giảng dạy môn “Phát triển ứng dụng IoT”- Võ Việt Dũng

Trang tài liệu chính thức của thư viện UniversalTelegramBot

Wokwi Documentation cho mô phỏng ESP32 và Wifi